

Parallel & Distributed Computing



SPEEDUP AND SCALABILITY IN PARALLEL COMPUTING



Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

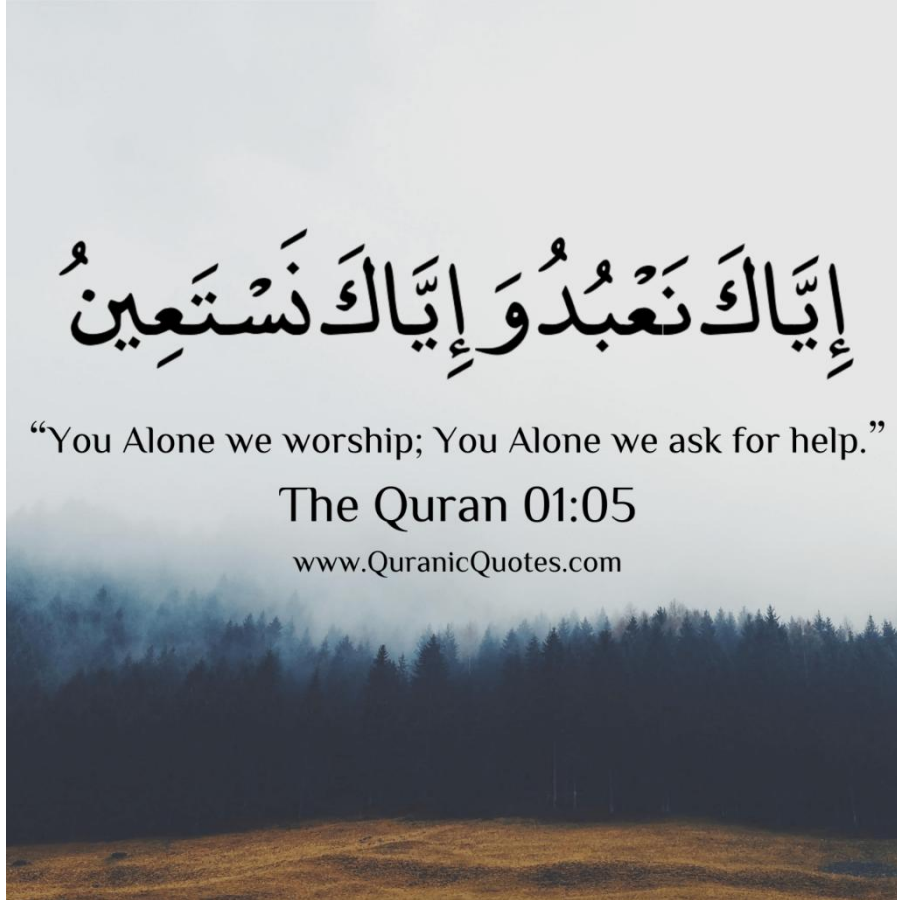
2

إِيَّاكَ نَعْبُدُ وَإِيَّاكَ نَسْتَعِينُ

“You Alone we worship; You Alone we ask for help.”

The Quran 01:05

www.QuranicQuotes.com



Lecture Outline

3

- Why Study Speedup and Scalability?
- Scalability
- Dimensions of Scalability
- Scaling Approaches
- Issues of Scalability
- Introduction to Amdahl's Law
- Practice Examples
- Limitations of Amdahl's Law



Why Study Speedup and Scalability?

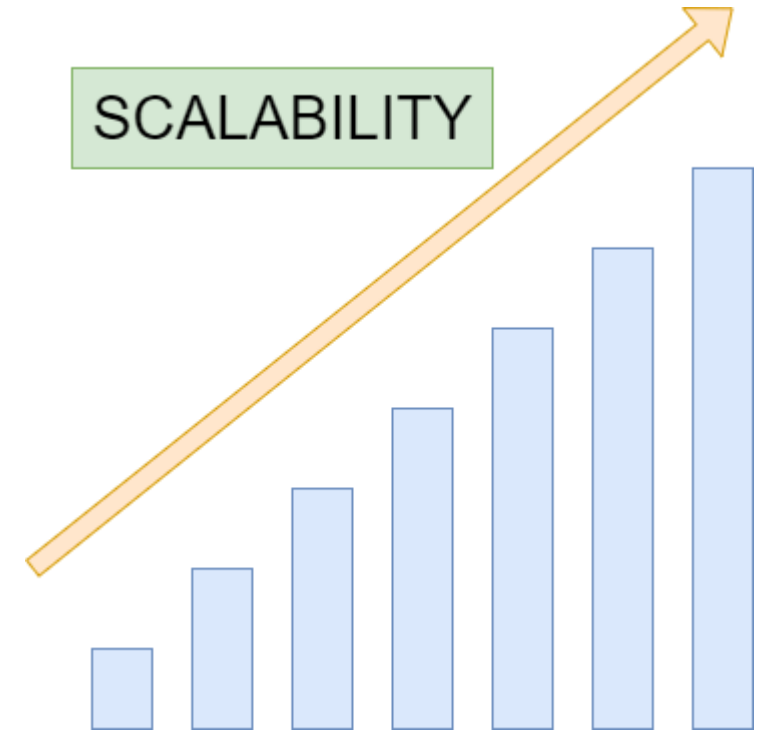
4

- How do we know if parallelization is worth it?
- Does adding more processors always mean faster execution?
- Helps in:
 - ▣ Evaluating performance improvements
 - ▣ Identifying bottlenecks in parallel programs
 - ▣ Deciding hardware cost vs. performance tradeoff
- Example:
 - ▣ Sorting a dataset of 1M records on 1 vs. 4 processors

Scalability

5

- ❑ Scalability is the property of a system to handle a **growing amount of work** by adding resources to the system.
- ❑ A system is described as scalable if it will remain effective when there is a significant increase in the number of resources and the number of users



Dimensions of Scalability (Overview)

6

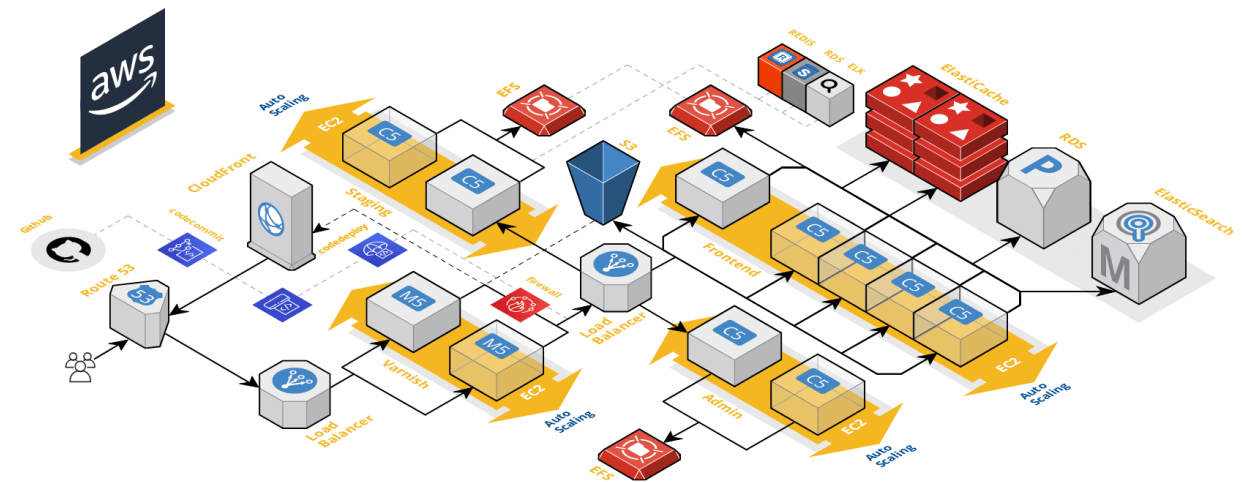
- Scalability is multi-dimensional, not just about adding processors.
- Key types:
 - ▣ Physical/Load
 - ▣ Administrative
 - ▣ Functional
 - ▣ Geographic
 - ▣ Generation
 - ▣ Heterogeneous



Physical / Load Scalability

7

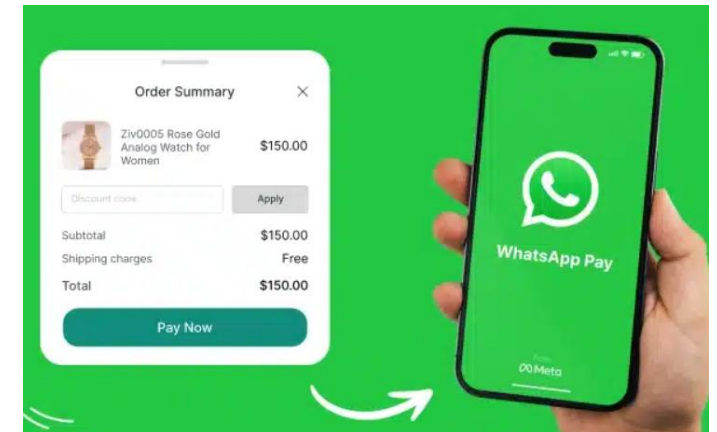
- Ability to handle more users or workload by adding resources (servers, CPUs, memory).
- Must be easy to add/remove resources.
- Example:
 - ▣ Web servers in cloud: add more instances when traffic spikes.
 - ▣ Netflix adding more servers to support peak streaming demand.



Administrative & Functional Scalability

8

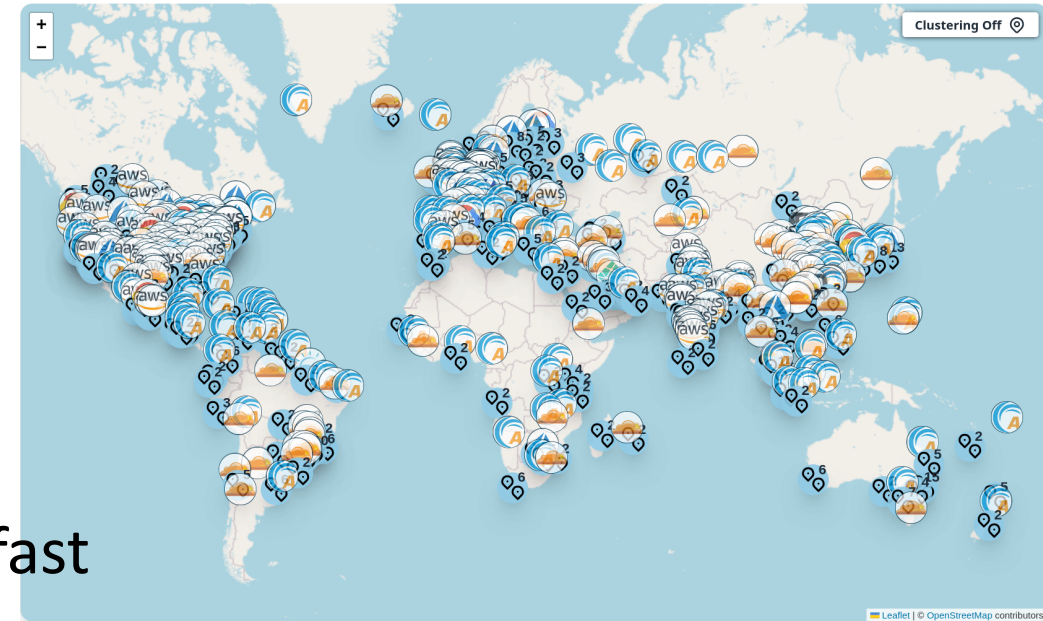
- **Administrative Scalability**
- Ability to support an increasing number of organizations/users.
- Example: Gmail supports billions of users without central bottlenecks.
- **Functional Scalability**
- Ability to add new features without breaking existing ones.
- Example: WhatsApp added “Payments” feature without disrupting chat system.



Geographic Scalability

9

- Ability to expand system effectiveness across regions.
- Example:
 - E-commerce site scaling from Pakistan → global users.
 - CDNs (Content Delivery Networks) ensure fast response worldwide.
- Challenge: network latency, regulations, localization.



Generation & Heterogeneous Scalability

10

- **Generation Scalability**

- Ability to adopt new versions/generations of components.
- Example:
 - ▣ Data centers moving from HDD → SSD → NVMe without redesigning system.

- **Heterogeneous Scalability**

- Ability to mix and integrate components from different vendors.
- Example:
 - ▣ Cloud-native apps using AWS + Azure + Google APIs in same system.

Scaling Approaches

11

- Two main approaches to improve system scalability:
 - ▣ **Scale Up** (Vertical Scaling):
Increase capacity of a single machine.
 - ▣ **Scale Out** (Horizontal Scaling):
Add more machines to share the load.
- Both aim to improve performance and handle larger workloads but differ in strategy.

Scale Up



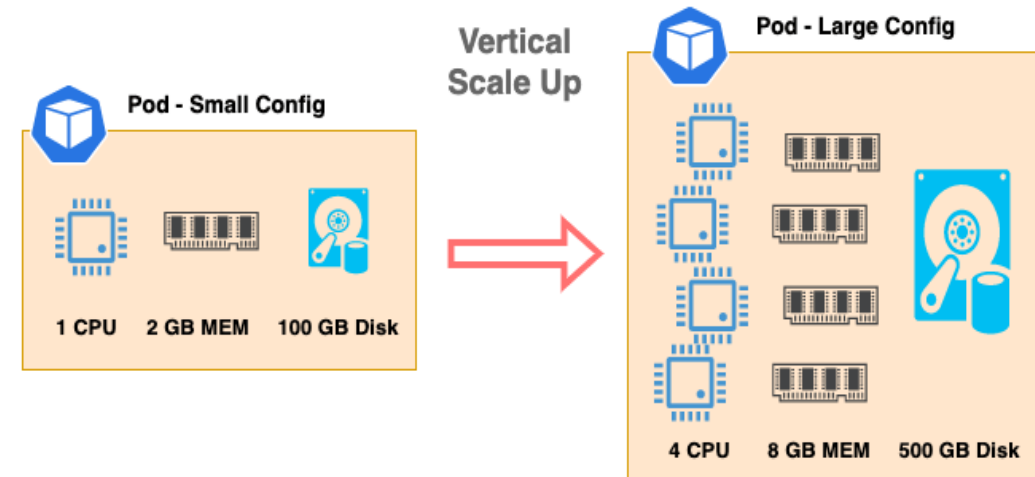
Scale Out



Scale Up (Vertical Scaling)

12

- ❑ Enhance a single node by adding more CPU, memory, or faster storage.
 - ▣ Simpler architecture.
 - ▣ Works well for small to medium workloads.
 - ▣ Limited by hardware capacity.
- ❑ Examples:
 - ▣ Upgrading from 8-core → 32-core CPU.
 - ▣ Adding more RAM to a database server.
- ❑ Pros:
 - ▣ Easy to implement, no app redesign.
- ❑ Cons:
 - ▣ Expensive, hardware upgrade limits, single point of failure.



Scale Up (Vertical Scaling)

13

Non Scalable: Single application server doing all parallel data processing or document processing. When maximum ram/hdd/cpu reaches, it could not process any more jobs or you would see system stops responding to job requests.



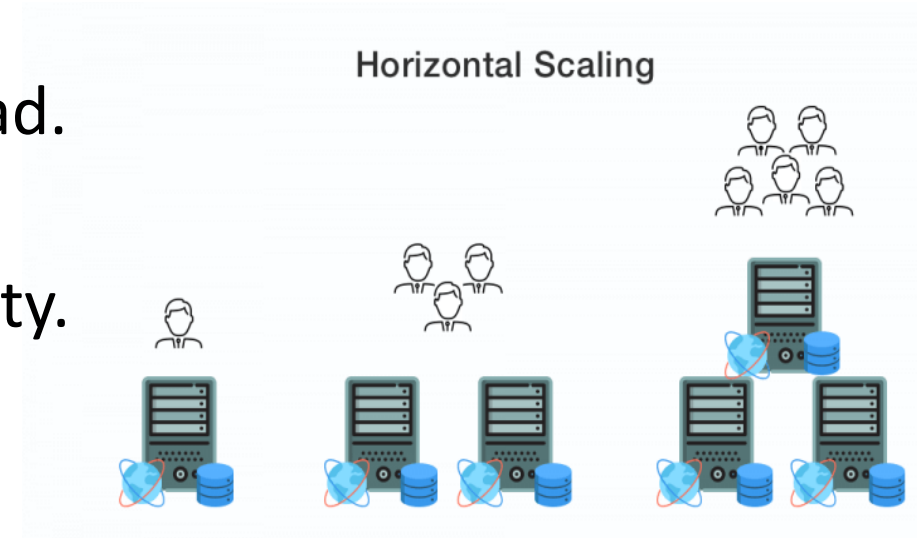
Vertically Scalable: Increase CPU, RAM & HDD depending on the need, so that single server can handle all the load.



Scale Out (Horizontal Scaling)

14

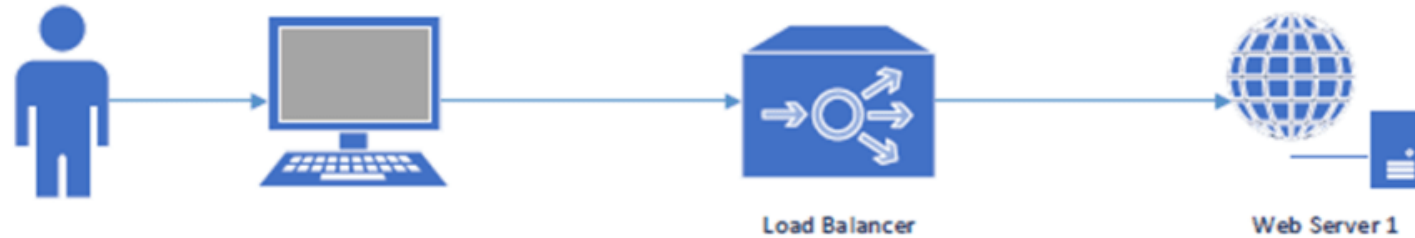
- ❑ Add more servers (nodes) to distribute workload.
- ❑ Requires distributed systems.
- ❑ Supports massive workloads and high availability.
- ❑ Examples:
 - ▣ Google search: thousands of commodity servers.
 - ▣ E-commerce sites: add more web servers behind load balancer.
- ❑ Pros:
 - ▣ Cost-effective, redundancy, virtually unlimited scalability.
- ❑ Cons:
 - ▣ Complexity (synchronization, consistency), software redesign needed.



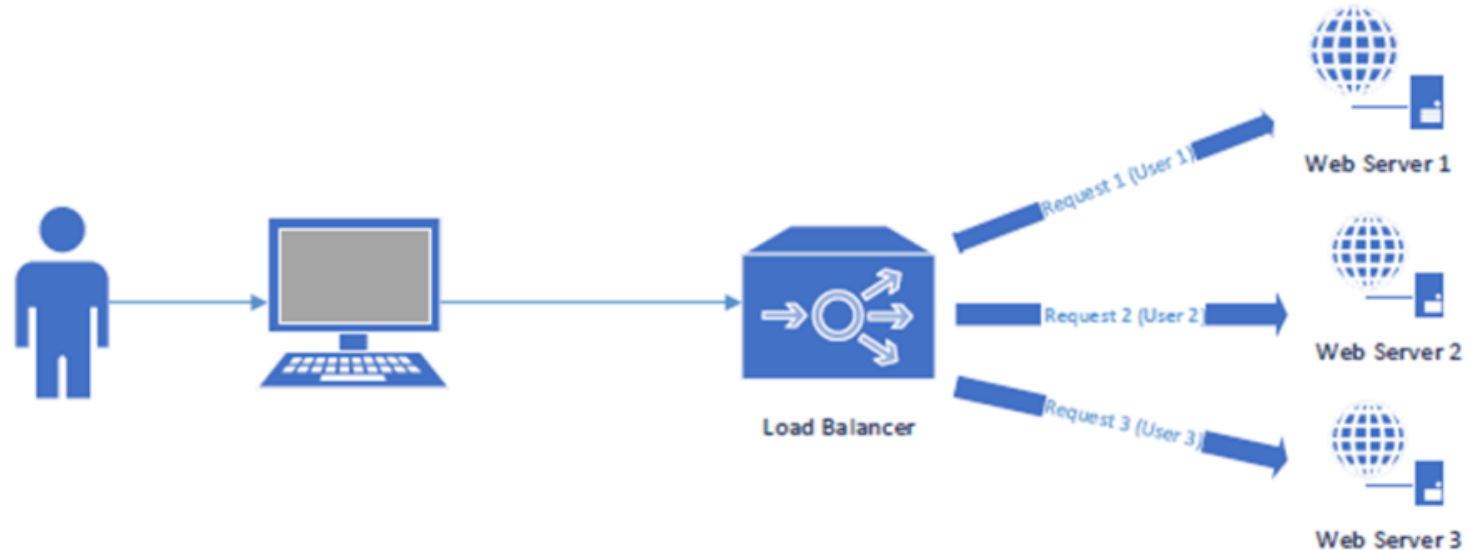
Scale Out (Horizontal Scaling)

15

Non Scalable: Single Server serving all the users. If the server cannot handle large number of users, it will not be able serve the subsequent requests.



Horizontally Scalable: Multiple Servers available to all the users. Depending on the load on each servers, request will be served by allocated server.



Issues of Scalability (Overview)

16

- Scaling a system introduces challenges beyond just adding processors or servers.
- Key issues:
 - ▣ Controlling the cost of physical resources
 - ▣ Controlling the performance loss
 - ▣ Preventing software resources from running out
 - ▣ Avoiding performance bottlenecks

Cost of Physical Resources & Performance Loss

17

□ **Controlling Cost of Physical Resources**

- ▣ Scaling out (more servers) or scaling up (bigger servers) costs money.
- ▣ Commodity servers may be cheaper, but networking & management costs rise.
- ▣ Example:
 - Cloud providers charge per instance; doubling servers may double cost without doubling performance.

□ **Controlling Performance Loss**

- ▣ Adding resources doesn't always equal faster execution.
- ▣ Synchronization, communication, and coordination overhead reduce performance.
- ▣ Example:
 - In distributed databases, replication improves reliability but increases latency.

Software Resource Exhaustion

18

❑ Preventing Software Resources Running Out

❑ Some software has limits on:

- Number of open file descriptors, sockets, or threads.
- Memory addressing (32-bit vs. 64-bit).

❑ Poorly designed applications may “hit ceilings” before hardware is fully utilized.

❑ Example:

- Web server crashes when concurrent connections exceed OS limit.

Performance Bottlenecks

19

□ Avoiding Bottlenecks

- A single bottleneck can limit system scalability, no matter how many processors are added.
- Typical bottlenecks:
 - Disk I/O (slow storage).
 - Network bandwidth/latency.
 - Lock contention in parallel programs.
- Example: One slow database node can slow down an entire e-commerce site.
- **Scalability = eliminating bottlenecks, not just adding resources.**

Introduction to Amdahl's Law

20

- *Amdahl's law* is a formula which gives the *theoretical speedup* in latency of the execution of a *task* at *fixed workload* that can be expected of a system whose *resources are improved* (scalability).
- It is named after computer scientist Gene Amdahl and was presented at the AFIPS Spring Joint Computer Conference in 1967.
- Amdahl's law is often used in parallel computing to predict the theoretical speedup when using multiple processors.

Concept of Speedup

21

- Speedup measures the improvement in execution time when using multiple processors compared to a single processor.

$$S(N) = \frac{T(1)}{T(N)}$$

- Where:
 - ▣ $T(1)$ = execution time with 1 processor
 - ▣ $T(N)$ = execution time with N processors
- **Key Points:**
 - ▣ $S(N) = N \rightarrow$ *Linear Speedup* (ideal but rarely achieved).
 - ▣ $S(N) < N \rightarrow$ *Sublinear Speedup* (common in real systems due to overhead).
 - ▣ $S(N) > N \rightarrow$ *Superlinear Speedup* (rare, occurs in special cases like caching effects).

Amdahl's Law

22

- Amdahl's law can be formulated in the following way:

$$S_{\text{latency}}(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

where

- S_{latency} is the theoretical speedup of the execution of the whole task;
- s is the speedup of the part of the task that benefits from improved system resources;
- p is the proportion of execution time that the part benefiting from improved resources originally occupied.

Amdahl's Law Serial Portion ($1 - P$)

23

- Always executed sequentially → cannot be parallelized.
- Examples in real systems:
 - ▣ Program initialization
 - ▣ I/O operations (disk access, user input)
 - ▣ Control/decision-making code
 - ▣ Synchronization overhead
- **Key Idea:** The serial portion becomes the **bottleneck** and sets the upper limit for performance.

Amdahl's Law Parallel Portion (P)

24

- Represents code that can be distributed across multiple processors.
- Examples:
 - ▣ Arithmetic operations in matrix multiplication
 - ▣ Processing different pixels in an image
 - ▣ Sorting data chunks in parallel before merging
- Benefit:
 - ▣ As N increases, execution time of the parallel part decreases (P/N).
- Limitation:
 - ▣ Diminishing returns if serial portion dominates.

Amdahl's Law Example 1

- For example, if a program needs **20 hours** using a **single processor** core, and a **particular part of the program** which takes one hour to execute cannot be parallelized.
- While the remaining **19 hours** [$19/20=0.95$] ($p = 0.95$) **95%** of execution time can be **parallelized**.
- But **0.05 (1 hour) 5%** part of the program can't be parallelized.

Amdahl's Law

- Then regardless of how many processors are devoted to a parallelized execution of this program, the minimum execution time cannot be less than that **critical one hour**.
- In the previous example 95% portion of process is subject to speed up, p will be 0.95 (**$p=0.95$**), and the portion which will get benefit of speed up twice of original then **$s=2$** .

Amdahl's Law

- So by using Amdahl's law overall speed up will be:
- Formula: $S = 1 / (1-p) + (p/s)$
- By putting values: $S = 1 / (1-0.95) + (0.95/2)$
- So the speed up (S): $S = 1 / (0.05) + (0.475) \Rightarrow 1 / 0.525 \Rightarrow \mathbf{1.907}$

Amdahl's Law Example 2 & 3

Suppose that we are considering an enhancement that runs 10 times faster than the original machine but is usable only 40% of the time. What is the overall speedup gained by incorporating the enhancement?

$$S_e = 10$$

$$F = 40/100 = 0.4$$

$$S_o = 1 / ((1 - F) + F/S_e)$$

$$= 1 / (0.6 + 0.4/10)$$

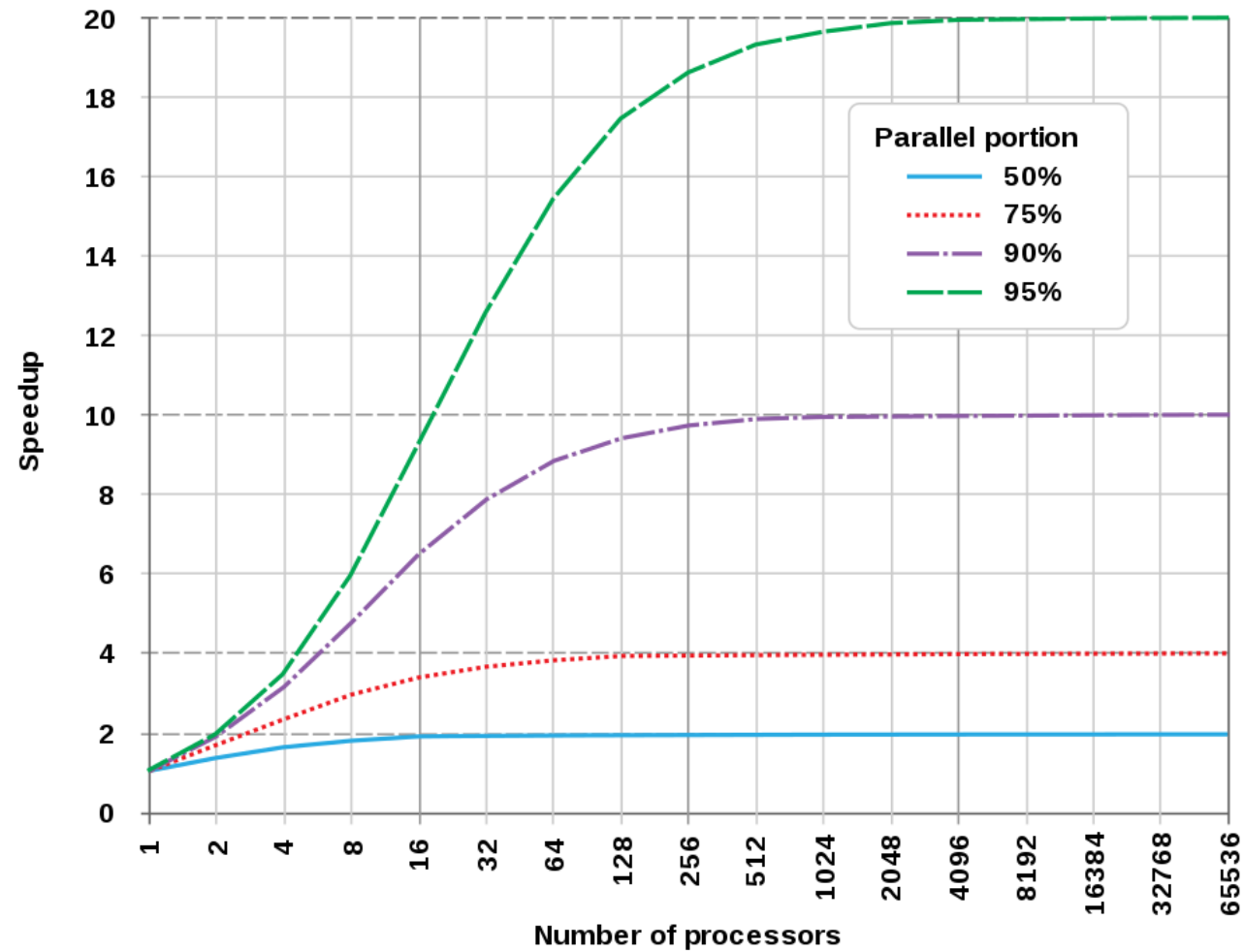
$$= 1/0.64 = 1.56.$$

Objective: Speedup Floating point square root operation. Which is better: Design1 or Design2?

Design 1: The operation uses FPSQR hardware. FPSQR is responsible for 20% of the square root execution time. Speedup this component by a factor of 10.

Design 2: Make all the floating point (FP) instructions run faster. How much faster? 2 times faster. Fp instructions are responsible for 50% of the square root execution time.

Amdahl's Law



Real-World Scenarios

□ **Web Servers:**

- ▣ Handling client requests in parallel.
- ▣ Logging & database locks remain serial.

□ **Scientific Simulations:**

- ▣ Heavy numerical computations parallelized.
- ▣ Data setup, pre/post-processing remain serial.

□ **Computer Games:**

- ▣ Graphics pipeline (shading, rendering) highly parallel.
- ▣ Game logic, physics engine, AI remain partly serial.

Real-World Scenarios

□ **Web Servers:**

- ▣ Handling client requests in parallel.
- ▣ Logging & database locks remain serial.

□ **Scientific Simulations:**

- ▣ Heavy numerical computations parallelized.
- ▣ Data setup, pre/post-processing remain serial.

□ **Computer Games:**

- ▣ Graphics pipeline (shading, rendering) highly parallel.
- ▣ Game logic, physics engine, AI remain partly serial.

Limitations of Amdahl's Law

- ❑ **Fixed workload assumption:** Ignores cases where workload grows with more processors.
- ❑ **Ignores communication & synchronization overhead.**
- ❑ Assumes equal performance across processors (not always true).
- ❑ **Alternative:** *Gustafson's Law* – better for scalable workloads (supercomputers, big data).

Thanks

33

*"If your actions inspire
others to dream more,
learn more, do more and
become more, you are a
leader."*

John Quincy Adams,
6th President of the United States